SIMATS ENGINEERING

CO1 - ASSESSMENT TOOL 1

Analytical Problem Solving

Name of the Student	SHAM S.
Reg. No	192325076

COURSE NAME: Deep Learning **COURSE CODE: MLA0403**
FACULTY : Dr.Arul Raj A.M

COURSE OUTCOME COVERED

CO 1: Learning Algorithms, Maximum Likelihood Estimation (MLE), Building Machine Learning Algorithms, Neural Networks, Artificial Neurons, Perceptron, Multilayer Perceptron (MLP), Forward Propagation, Backpropagation Algorithm, Gradient Descent, Stochastic Gradient Descent (SGD), Mini-Batch Gradient Descent, Curse of Dimensionality. (BTL-4)

CO1 Weightage: 15%

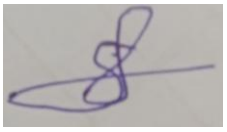
Assessment Tool Weightage: 35%

Duration: 30 minutes

Marks: 25

Code of Conduct:

I SHAM S. - 192325076 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.



Signature of the student: **SHAM S.**
Name of the student:

QUESTIONS:05 **Total Marks:25**

Analytical Problem Solving

1. Learning Algorithms (Optimization Behavior)

A machine learning model is trained using gradient descent on a convex loss function. Initially, the learning rate is set very high, and later it is gradually decreased.

Question:
Analyze how the choice of a high initial learning rate followed by decay affects convergence. Under what conditions can this strategy outperform a constant learning rate? Discuss possible risks and justify with reasoning.

Answer :

A **high initial learning rate** allows gradient descent to make **large parameter updates**, helping the model quickly move toward the minimum of the convex loss function. As the learning rate is **gradually decreased (learning rate decay)**, the updates become smaller, enabling the algorithm to converge more accurately to the optimal solution.

Effect on Convergence

- **High initial learning rate:**
    - Speeds up early training.
    - Covers large distances in fewer iterations.
    - Helps escape flat regions or plateaus faster.
- **Gradually decreasing learning rate:**
    - Reduces oscillations near the minimum.

- Allows fine-tuning of parameters.

- Improves convergence stability and accuracy.

When This Strategy Outperforms a Constant Learning Rate

This approach performs better than a constant learning rate when:

- The optimization starts far from the optimum.

- The dataset is large and requires faster initial learning.

- The loss function is smooth and convex.

- High speed is needed initially, followed by precise convergence.

A constant learning rate may either:

- Be **too small**, causing slow convergence.

- Be **too large**, preventing convergence or causing oscillations.

Possible Risks

- If the initial learning rate is **too high**, the algorithm may:

    - Overshoot the global minimum.

- Oscillate around the minimum.

- Diverge instead of converging.

- If the learning rate decreases **too quickly**, learning may stop before reaching the optimal solution.

- If it decreases **too slowly**, convergence may remain inefficient.

Justification

Using a **high initial learning rate followed by decay** combines the advantages of **fast learning** in the early stages and **stable, accurate convergence** in the later stages. When the decay schedule is chosen appropriately, this strategy usually converges **faster and more accurately** than using a fixed learning rate throughout training.

2. Maximum Likelihood Estimation (MLE)

Suppose you are given a dataset generated from a Gaussian distribution with unknown mean μ and known variance σ^2

Question:
Derive the Maximum Likelihood Estimator for μ, and analytically explain how the estimate changes when:

- The dataset contains outliers
- The sample size increases
  What does this imply about the robustness of MLE?

  Suppose the dataset is:

$$X = \{x_1, x_2, \dots, x_n\}$$

  where each x_i is independently drawn from a Gaussian distribution:

$$X_i \sim N(\mu, \sigma^2)$$

  Here, μ is unknown and σ^2 is known.

  1. Derivation of Maximum Likelihood Estimator (MLE) for μ

The likelihood function is:

$$L(\mu) = \prod_{i=1}^{n} \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(\quad -\frac{(x_i - \mu)^2}{2\sigma^2} \right)$$

Taking the logarithm,

$$\log L(\mu) = -\frac{n}{2}\log(2\pi\sigma^2) - \frac{1}{2\sigma^2}\sum_{i=1}^{n}(x_i - \mu)^2$$

Differentiate with respect to μ:

$$\frac{d\log L}{d\mu} = \frac{1}{\sigma^2}\sum_{i=1}^{n}(x_i - \mu)$$

Set the derivative equal to zero:

$$\sum_{i=1}^{n}(x_i - \mu) = 0$$
$$\sum_{i=1}^{n} x_i - n\mu = 0$$

Therefore,

$$\boxed{\hat{\mu}_{MLE} = \frac{1}{n}\sum_{i=1}^{n} x_i}$$

Thus, the **MLE of the mean is the sample mean**.

2. Effect of Outliers

An outlier is a value that is much larger or smaller than the other observations.

Since

$$\hat{\mu} = \frac{x_1 + x_2 + \cdots + x_n}{n},$$

even a single extreme value changes the average significantly.

Example:

Without outlier:

Data = {5, 6, 7, 8, 9}

$$\hat{\mu} = 7$$

With outlier:

Data = {5, 6, 7, 8, 100}

$$\hat{\mu} = \frac{126}{5} = 25.2$$

The estimate shifts from **7 to 25.2**, showing that the MLE (sample mean) is **highly sensitive to outliers**.

3. Effect of Increasing Sample Size

As the number of samples $n$increases:

- The sample mean becomes closer to the true population mean.
- The variance of the estimator decreases.

$$\boxed{\text{Var}(\hat{\mu}) = \frac{\sigma^2}{n}}$$

Hence,

- Larger sample size → Smaller variance.
- Estimate becomes more accurate and stable.
- By the **Law of Large Numbers**, the sample mean converges to the true mean.

4. Implication on Robustness of MLE

Advantages

- Efficient estimator for Gaussian data.
- Unbiased estimator.
- Consistent (approaches the true mean as sample size increases).
- Minimum variance among unbiased estimators under normal distribution.

Limitations

- Highly sensitive to outliers because it uses the arithmetic mean.
- A few extreme observations can significantly affect the estimate.

3. Building a Machine Learning Algorithm (Model Selection)

You are designing a classification algorithm for a dataset with 10,000 features but only 500 training samples.

Question:
Analyze the challenges involved in training such a model. Propose a strategy (algorithm pipeline) to handle this situation and justify each step (e.g., feature selection, regularization, dimensionality reduction). Explain how your approach mitigates overfitting.

Challenges

1. **Curse of Dimensionality**
    - The number of features is much larger than the number of samples.
    - The model becomes complex and difficult to learn.
2. **Overfitting**
    - The model may memorize the training data instead of learning patterns.
    - High training accuracy but low test accuracy.
3. **High Computational Cost**
    - More features require more memory and training time.
4. **Irrelevant and Redundant Features**
    - Many features may not contribute to prediction.
    - Noise reduces model performance.
5. **Poor Generalization**
    - With only 500 samples, the model may fail to perform well on unseen data.

Proposed Machine Learning Pipeline

Step 1: Data Preprocessing

- Handle missing values.
- Normalize or standardize the features.
- Remove duplicate or noisy data.

Justification: Ensures all features are on the same scale and improves training stability.

Step 2: Feature Selection

Use techniques such as:

- Chi-Square Test
- Mutual Information
- Recursive Feature Elimination (RFE)
- L1 (Lasso) Feature Selection

Reduce **10,000 features to a few hundred important features**.

Justification:

- Removes irrelevant features.
- Reduces model complexity.
- Speeds up training.
- Lowers overfitting.

Step 3: Dimensionality Reduction

Apply **Principal Component Analysis (PCA)**.

Example:

10,000 Features → 100 Principal Components

Justification:

- Removes feature correlation.
- Preserves most data variance.
- Reduces computational cost.
- Simplifies the learning problem.

Step 4: Choose a Regularized Classifier

Examples:

- Logistic Regression with **L1/L2 Regularization**
- Support Vector Machine (Linear SVM)
- Regularized Neural Network

Justification:
Regularization penalizes large model weights and prevents the model from fitting noise.

Step 5: Cross-Validation

Use **K-Fold Cross Validation** (e.g., 5-fold or 10-fold).

Justification:

- Makes better use of only 500 samples.
- Produces reliable performance estimates.
- Helps detect overfitting.

Step 6: Hyperparameter Tuning

Use:

- Grid Search
- Random Search

Tune parameters such as:

- Regularization strength (λ)
- Number of selected features
- Number of PCA components

Justification:
Finds the best model while avoiding excessive complexity.

Step 7: Final Evaluation

Evaluate using:

- Accuracy
- Precision
- Recall
- F1-Score
- ROC-AUC

How This Approach Mitigates Overfitting

Technique	How it Reduces Overfitting
Feature Selection	Removes irrelevant and noisy features.
PCA	Reduces dimensionality while preserving important information.
Regularization (L1/L2)	Penalizes overly complex models and limits large weights.
Cross Validation	Ensures the model generalizes well to unseen data.

Technique	How it Reduces Overfitting
Hyperparameter Tuning	Selects the optimal model complexity.

Conclusion

For a dataset with **10,000 features and only 500 training samples**, the main challenge is **overfitting due to high dimensionality**. An effective pipeline is:

Data Preprocessing → Feature Selection → PCA → Regularized Classifier → Cross Validation → Hyperparameter Tuning → Model Evaluation

This approach removes irrelevant features, reduces dimensionality, controls model complexity through regularization, and validates performance using cross-validation. As a result, the model generalizes better to unseen data and significantly reduces the risk of overfitting.

4. Neural Networks (Multilayer Perceptron - MLP)

Consider a Multilayer Perceptron with one hidden layer using sigmoid activation functions.

Question:
Explain analytically why adding more hidden neurons increases the representational power of the network. Then, reason why this does not always lead to better performance. Support your answer using concepts like overfitting and generalization.

1. Why Adding More Hidden Neurons Increases Representational Power

Each hidden neuron learns a different feature or pattern from the input data. Adding more neurons allows the network to represent more complex functions.

- With **few hidden neurons**, the MLP can learn only simple decision boundaries.
- With **more hidden neurons**, the network can combine many nonlinear transformations, enabling it to model highly complex relationships between inputs and outputs.

Thus, increasing the number of hidden neurons increases the **capacity (representational power)** of the network, allowing it to approximate more complex functions (Universal Approximation Theorem).

2. Why More Hidden Neurons Do Not Always Improve Performance

Although more neurons increase learning capacity, they do not always improve prediction accuracy.

(a) Overfitting

- A network with many hidden neurons may memorize the training data instead of learning general patterns.
- It performs very well on the training set but poorly on unseen test data.

Result: High training accuracy but low testing accuracy.

(b) Poor Generalization

Generalization is the ability of a model to make accurate predictions on new, unseen data.

- An overly complex MLP captures both useful patterns and random noise.
- As a result, it cannot generalize well to new data.

(c) Increased Computational Cost

More hidden neurons increase:

- Number of weights and biases
- Memory usage
- Training time

This makes the model slower and more computationally expensive.

(d) Vanishing Gradient Problem

Since the hidden layer uses the **sigmoid activation function**, increasing the number of neurons in deeper networks can lead to very small gradients during backpropagation.

This slows learning and may prevent the network from converging efficiently.

3. Overfitting vs. Generalization

More Hidden Neurons	Effect
Increased model capacity	Learns more complex patterns
Too many neurons	Memorizes training data (overfitting)
Good balance of neurons	Learns useful patterns and generalizes well
Excessive complexity	Poor performance on unseen data

Conclusion

Adding more hidden neurons in an MLP **increases its representational power** because the network can learn more complex nonlinear relationships. However, **more neurons do not always improve performance**. Excessive neurons increase model complexity, causing **overfitting**, reduced **generalization**, higher computational cost, and possible training difficulties with sigmoid activation. Therefore, the optimal number of hidden neurons should be chosen carefully using techniques such as **cross-validation, regularization, and early stopping** to achieve good generalization.

5. Backpropagation, SGD & Curse of Dimensionality

A deep neural network is trained using Stochastic Gradient Descent (SGD) on a very high-dimensional dataset.

Question:
Analyze how the **curse of dimensionality** impacts:

- Gradient updates in backpropagation
- Convergence speed of SGD
- Model generalization

Propose at least two techniques to overcome these issues and justify them analytically.

1. Impact on Gradient Updates in Backpropagation

During backpropagation, gradients are computed for every weight in the network.

In high-dimensional data:

- The network contains a very large number of parameters.
- Many features are irrelevant or noisy, producing unstable or noisy gradients.
- Gradients may become **very small (vanishing gradients)** or **very large (exploding gradients)**.
- Weight updates become less effective, making optimization difficult.

Result: Gradient updates become unstable, leading to slower and less efficient learning.

2. Impact on Convergence Speed of SGD

Stochastic Gradient Descent (SGD) updates model parameters using one or a few training samples at a time.

In high-dimensional datasets:

- More parameters require more computations per iteration.
- The optimization landscape becomes more complex.
- SGD may oscillate or take many iterations to reach the optimum.
- More epochs are required for convergence.

Result: SGD converges more slowly and may get stuck in poor local minima or saddle points.

3. Impact on Model Generalization

When the number of features is much larger than the number of training samples:

- The network can memorize the training data.

- It learns noise instead of meaningful patterns.
- Training accuracy becomes high, while testing accuracy becomes low.

This leads to **overfitting**, reducing the model's ability to generalize to unseen data.

4. Techniques to Overcome These Issues

Technique 1: Dimensionality Reduction (PCA)

- Apply **Principal Component Analysis (PCA)** to reduce thousands of features into a smaller set of principal components while retaining most of the important information.

Justification:

- Removes redundant and correlated features.
- Reduces the number of parameters.
- Makes gradient updates more stable.
- Improves SGD convergence.
- Reduces overfitting.

Technique 2: Regularization (L1/L2 or Dropout)

- **L1/L2 Regularization** adds a penalty to large weights.
- **Dropout** randomly deactivates neurons during training.

Justification:

- Prevents the network from relying on specific features.
- Reduces model complexity.
- Improves generalization.
- Prevents overfitting.

Additional Techniques

- **Feature Selection:** Remove irrelevant features before training.
- **Early Stopping:** Stop training when validation error begins to increase.
- **Batch Normalization:** Stabilizes gradients and accelerates convergence.
- **Increase Training Data / Data Augmentation:** Provides more information for learning and improves generalization.

Summary Table

Aspect	Impact of Curse of Dimensionality
Gradient Updates	Noisy, unstable, vanishing/exploding gradients
SGD Convergence	Slower convergence and more iterations required
Model Generalization	High risk of overfitting and poor performance on unseen data

Conclusion

The **curse of dimensionality** makes deep neural networks difficult to train by producing unstable gradient updates, slowing SGD convergence, and increasing the risk of overfitting. Techniques such as **dimensionality reduction (PCA)** and **regularization (L1/L2 or Dropout)** effectively reduce model complexity, improve gradient stability, speed up convergence, and enhance generalization. Additional methods like **feature selection, early stopping, batch normalization, and increasing training data** further improve the model's ability to perform well on unseen data.